PRODUCT SPECIFICATION DOCUMENT

Project Name:	Hybrid Custom Web Browser (Desktop & Mobile)
Document Version:	v1.0.0
Target Platforms:	Windows, macOS, Linux, Android, iOS
Core Capabilities:	Dynamic HTML Execution, Multi-protocol Media Interception & Extraction

1. Executive Concept

The Hybrid Custom Web Browser is a dual-engine sandbox browser application designed for desktop and mobile environments. The core ethos of this application is twofold: absolute user sovereignty over local code execution and autonomous media downloading capability. Unlike standard commercial browsers that isolate local files via restrictive CORS (Cross-Origin Resource Sharing) boundaries and strip media sniffing utilities to comply with ecosystem constraints, this application delivers an engine capable of dynamically rendering hot-pasted standalone HTML packages while serving as a network-level multi-threaded asset extraction manager.

The architecture treats pasted code segments as sandboxed virtual documents while deploying background asynchronous sniffers that monitor system-level sockets and framework-level rendering webviews to extract raw streams, chunked manifests, and responsive media tags.

2. Core Technical Features

2.1 Dynamic Real-Time HTML Injection & Persistent Parsing

- **Code Sandboxing:** Provides a unified viewport interface allowing raw HTML strings or structural markup trees to be pasted directly into an interceptor input. The browser compiles these entries natively without disk storage overhead by parsing strings using memory buffers via localized data frames.
- **Data URI Compilation:** Standard injection structures utilize dynamic base64 conversion blocks. For deep multi-asset projects, the application translates text chunks using the formula: $S_{uri} = ext{"data:text/html;base64,"} + ext{Base64}(S_{html})$.
- **Hybrid Storage Cache:** Integrated SQLite and indexed local file subsystems persist injected structures on-disk when explicitly flagged by the user, generating unique internal routes such as `local://hash_id/index.html` to allow subsequent sessions to re-render saved projects with cross-reference accessibility.

2.2 Autonomous Network-Level Media Sniffing & Interception

- **Hooked Resource Sniffing:** Intercepts outbound web requests generated by active tabs before communication handshakes resolve. It actively checks metadata headers for structural matching across image layers and network stream protocols.
- **Adaptive Content Type Detection:** Scans responding headers for target MIME definitions (e.g., `video/mp4`, `video/webm`, `image/webp`, `application/x-mpegURL`).
- **Manifest Splicing (HLS/DASH):** Recognizes index configurations containing stream structures (`.m3u8` or `.mpd` threads). The browser automatically intercepts the target chunk sequences, hooks an internal instance of a download utility, and serializes the dynamic stream blocks directly into a singular user file container.

> **Universal Interception Mechanism:** By binding code listeners directly into the application network engine wrappers (Electron `webRequest` API and Flutter `InAppWebView` framework), media payloads are parsed at the runtime kernel layer before modern obfuscation layers can intercept or mask the sources.

2.3 Standard Web Browsing Package

- Fully-featured browser UI featuring a unified address navigation bar, backward/forward navigation nodes, session-tracked tabs, bookmarks, history trackers, and full support for secure cryptographic handshakes (TLS/SSL).

3. Product Architectural Scope

3.1 In-Scope Capabilities

- Cross-platform availability utilizing highly performant framework engines tailored for specific host environments (Desktop vs. Mobile).
- Dynamic download dashboard featuring synchronous multi-threaded network connection streams to drastically optimize down-pipe speeds.
- Execution layer for pasted HTML allowing full access to JavaScript rendering arrays, CSS canvas elements, and web graphics configurations.
- Localized file-system mirroring for offline-rendered components, ensuring relative stylesheet directories or embedded image pointers map smoothly.

3.2 Out-of-Scope Capabilities

- Cloud synchronizations or remote profile storage layers; all saved pages, downloaded binaries, and histories are kept strictly local to maximize privacy.

- Bypassing Digital Rights Management (DRM) components like Widevine or FairPlay. Encrypted stream payloads requiring secure decryption keys remain bounded to respect hardware runtime rules.

4. Cross-Platform Technology Stack

Target Platform	Framework / Core Stack	Key Packages & Infrastructure Dependencies
Desktop Browser (macOS, Win, Linux)	**Electron** **Node.js** **Chromium**	`session.webRequest` for network sniffing; `ffmpeg-static` for media chunk aggregation; `Better-SQLite3` for page management.
Mobile Browser (iOS & Android)	**Flutter** **Dart** **Native WebKit/Blink**	`flutter_inappwebview` for intercepting headers; `flutter_downloader` for background transfers; `sqflite` for local state retention.
Core Processing (Shared Utilities)	**JavaScript (ES12)** **FFmpeg C-Core**	Binary demuxing libraries; DOM parser selectors for image asset discovery.

5. Pipeline Workflow & Processing Model

The following processing sequencing outlines the workflow for media discovery and page transformation pipelines implemented across the structural components of the application:

1. **Initialization Node:** Webview renders target structural frames using platform layout viewports. Custom JS files are continuously injected as content scripts upon frame handshakes.
2. **Network Evaluation:** Network communication packets pass through processing gates. If an active data packet returns structural tags such as `Content-Type: application/vnd.apple.mpegurl`, a parsing interceptor breaks off the active link.
3. **Asset Reconstruction:** For fragmented streams, the background file manager uses multithreaded download threads to fetch stream elements, feeding them to an integrated execution block to compile an intact, standalone media file.